01 / INSTRUCTOR GUIDE

Teaching InvoiceAI

An instructor playbook for explaining the business problem, the .NET architecture, and the AI request flow to your students.

Audience assumption: students know basic C#, methods, classes, JSON, and HTTP. Explain new architecture terms as they appear. **Format:** two 90-minute sessions, plus optional extension work.

SAY THIS
This app helps a business keep customer records, prepare itemized invoices, track what has been paid, and see what is still owed. It also provides a chat assistant that can explain the app and use selected business operations. We will first understand the billing rules, then trace a request, and finally see how the AI uses those same rules.

What a student should be able to explain afterward

A student should be able to calculate an invoice total; predict valid status changes; identify the role of each project; trace one normal request and one AI request; and separate a model response from evidence that a business operation succeeded.

Your teaching principle
Repeat one familiar example as you add layers. Show a customer and a NGN 60,000 invoice before introducing CQRS, agents, tools, or middleware.

How to use this guide

Amber boxes are phrases you can say aloud. **Show** instructions identify what to demonstrate. **Ask / listen for** gives checks for understanding. The final pages contain exercises, answers, an assessment rubric, and exact code references.

The earlier technical walkthrough can serve as your reference during preparation. This guide focuses on teaching order, delivery, demonstrations, and student practice. It also stands on its own.

Prepared from the current InvoiceAiApp workspace on 17 October 2026. This is a proposed teaching plan based on static code review. No classroom demo, model call, database mutation, or test run was performed while creating it. NGN denotes Nigerian Naira; sample amounts and fixtures are instructional.

02 / INSTRUCTOR GUIDE

Plan two focused sessions

Use a predict, observe, explain cycle: ask students what should happen, run or inspect the example, then connect it to the code.

Session 1: the business application - 90 minutes

Minutes	Teach and demonstrate	Evidence of learning
0-10	Introduce the billing problem and the NGN 60,000 example.	Students calculate the total.
10-25	Create a customer and draft invoice in the normal UI.	Students identify inputs and output.
25-40	Explain domain rules and the invoice state machine.	Students predict valid transitions.
40-55	Map the six projects to responsibilities.	Students place a rule in the right layer.
55-75	Trace normal invoice creation through API, handler, domain, and save.	Students narrate the request path.
75-85	Explain queries and dashboard metrics.	Students distinguish counts from amounts.
85-90	Five-minute recap and exit question.	One student explains why Draft-to-Paid fails.

Session 2: the AI integration - 90 minutes

Minutes	Teach and demonstrate	Evidence of learning
0-10	Recap business rules and the normal request path.	Students recall the save boundary.
10-25	Walk through AiEndpoints and AgentCoordinator.	Students predict agent selection.
25-40	Explain IChatClient, function calling, tools, and data grounding.	Students distinguish model and application work.
40-55	Demonstrate prompts and compare execution modes.	Students identify the actual mode.
55-65	Explain evaluation, success flags, and evidence.	Students challenge a misleading score.
65-80	Complete the three short exercises.	Students submit paths, predictions, and calculations.
80-90	Teach-back, feedback, and next steps.	Students explain the app in their own words.

If students struggle with HTTP or dependency injection, split the sessions into shorter lessons and add a prerequisite exercise. For experienced students, shorten the UI tour and use that time to inspect validation and failure paths. The timings are a suggested plan, not a measured delivery duration.

03 / INSTRUCTOR GUIDE

Prepare a reliable classroom demonstration

Start both applications and confirm the business API works before depending on the chat experience.

Before the lesson

Use a training database with predictable records. Have the .NET 10 SDK and the configured SQL Server instance available. Verify the API database connection and the web app ApiSettings:BaseUrl. Decide whether you will teach with live Azure OpenAI or backend simulation, and tell students which one you are using.

```
# From the repository root, in separate terminals:
dotnet run --project src/InvoiceApp.Api --launch-profile http
dotnet run --project src/InvoiceApp.Web --launch-profile http
```

The checked-in HTTP profiles use API port 5231 and Web port 5100. Both profiles set Development. The web Development settings point to http://localhost:5231. Follow the actual console listening addresses if configuration has changed. Development Swagger is mapped at the API root. [T37:8] [T36:8] [T38:9] [T04:44]

Rehearse these five checks

1. The dashboard and customer list load real data. **2.** A new training customer can be created. **3.** A draft invoice saves with a calculated total. **4.** You can identify the current AI mode from configuration and logs. **5.** The three read-oriented prompts on page 13 produce the expected routes.

> **SAY THIS**
> The API and web interface are separate running applications. The browser sends its UI request to the web app. The web app calls the backend API. For a direct API demonstration, we can use Swagger without the MVC interface.

> **A fallback demonstration plan**
> If cloud access is unavailable, teach the coordinator and tool boundaries using backend simulation. If SQL Server is unavailable, use the paper fixtures and debugger/code walk instead of claiming that live database operations succeeded.

API startup attempts schema initialization and seeding, but catches failures and logs a warning. A running process therefore does not prove that the database is reachable. [T04:61]

04 / INSTRUCTOR GUIDE

Open with a business story

Keep one customer and one invoice visible while introducing technical concepts.

SAY THIS
Imagine a small design business called Lagos Design Studio. It has completed work for a customer and needs a record of what was supplied, how much it costs, when payment is due, and whether payment has been made. That record is the invoice. The customer record tells us who owes the money.

Line item	Quantity	Unit price	Line total
Design consultation	2	NGN 25,000	NGN 50,000
Brand review	1	NGN 10,000	NGN 10,000
Total invoice			**NGN 60,000**

Show the workflow before showing code

1. Create a customer such as Classroom Customer 01 with a unique training email. **2.** Create an invoice for that customer, using the class date and a due date 14 days later. **3.** Enter the two items above. **4.** Read the generated number and record it on the board. **5.** Open its details and ask students to verify the total and Draft status.

Use the actual generated invoice number throughout the demo. Do not assume a fresh database will generate INV-000001 or that the next number has no gaps. Keep the invoice in Draft until the state-machine activity.

ASK: If we change consultation quantity from 2 to 3 while the invoice is still editable, what should the total become?

LISTEN FOR: NGN 85,000: 3 x 25,000 plus 10,000. The total is derived from the items.

Connect the story to the domain objects

Customer: identity and contact details. **Invoice:** customer association, dates, status, and its items. **InvoiceItem:** description, quantity, unit price, and calculated LineTotal. Invoice.TotalAmount sums its item totals. [T03:7] [T01:23] [T02:13]

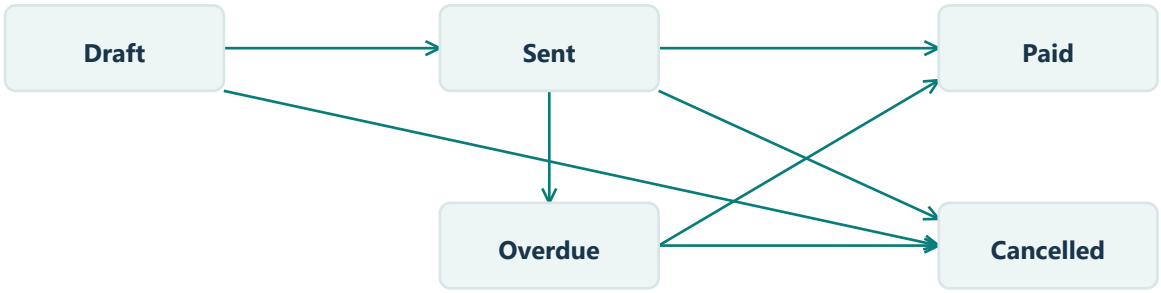
Keep the first explanation narrow
Teach the implemented arithmetic: quantity times unit price, summed across items. The inspected entity calculation has no separate tax, discount, or payment-ledger calculation. Introduce those only as possible future features.

05 / INSTRUCTOR GUIDE

Teach the rules as a state machine

A status is a controlled business condition, not a label the caller can change arbitrarily.

SAY THIS
The invoice begins as a Draft. We can send it, then mark it Paid or Overdue. We can cancel an eligible invoice, but we cannot move freely between every status. These restrictions live in the domain so every caller has to respect them.



Current status	Allowed different target statuses
Draft	Sent, Cancelled
Sent	Paid, Overdue, Cancelled
Overdue	Paid, Cancelled
Paid or Cancelled	None

Invoice.TransitionTo implements these transitions and accepts a request for the existing status as success. CanBeDeleted is true only for Draft. Invoice.Update also requires Draft. [T01:147] [T01:25] [T01:102]

A demonstration students can predict

Ask what should happen to Draft-to-Paid before attempting it through the business API. If the UI hides that option, show the domain test or use Swagger on the training record. Expect rejection. Then do Draft-to-Sent and Sent-to-Paid, inspecting the result after each step. Use another disposable Draft if demonstrating deletion.

ASK: Why is hiding a button insufficient to enforce this rule?

LISTEN FOR: Another client or an AI tool could call the backend. The domain must enforce the transition regardless of the caller.

Correct a common simplification
Do not say that every non-draft invoice can be cancelled. The current code does not allow Paid-to-Cancelled. It also does not automatically mark a stored status Overdue merely because a query sees a past due date.

06 / INSTRUCTOR GUIDE

Introduce architecture with responsibilities

Present the folders after students understand the workflow they exist to support.

Project	Explain it in one sentence	Example
InvoiceApp.Web	Receives browser actions, calls the API, and presents views.	InvoicesController and typed API clients.
InvoiceApp.Api	Accepts HTTP input and maps it to application requests.	InvoicesEndpoints and AiEndpoints.
InvoiceApp.Application	Coordinates a specific use case.	CreateInvoiceCommandHandler.
InvoiceApp.Domain	Owns invoice/customer rules and business objects.	Invoice.TransitionTo.
InvoiceApp.Infrastructure	Implements storage and external technical details.	EF repositories, Dapper SQL, numbering.
InvoiceApp.AI	Connects natural language to registered business operations.	Coordinator, agents, tools, evaluation, memory.

SAY THIS
Think of a small billing office: the web interface is the counter, the API is the intake desk, the application layer organizes the job, the domain is the rulebook, and infrastructure handles storage. The AI assistant is another way to request work. This analogy describes responsibilities; it does not mean each project runs as a separate service.

Distinguish execution from dependencies

At runtime a request can go from the API to a handler and then a repository. In the code design, application/domain contracts describe the storage operations needed, and Infrastructure implements them. Dependency injection selects the implementations. The model is not given the database connection string or arbitrary SQL access by this tool surface.

ASK: Where should "only Draft invoices can be deleted" live?

LISTEN FOR: In the domain as a business rule, with the handler checking it. The UI can reflect the rule, but cannot replace it.

ASK: Where should we change a SQL query for summary totals?

LISTEN FOR: The Infrastructure read repository. If we change the meaning of the metric, we should also review the contract, UI, tests, and documentation.

Use API Program.cs to show AddApplication, AddInfrastructure, and AddAgentFramework. Use the web Program.cs to show its HTTP clients. Stop before explaining every registration. [T04:30] [T05:31]

07 / INSTRUCTOR GUIDE

Trace an ordinary invoice request

Explain this path before introducing AI. It becomes the anchor for the rest of the lesson.

Browser -> MVC web action
The form supplies customer ID, dates, notes, and line items.

↓

Web typed client -> backend endpoint
InvoiceApiClient sends JSON to POST /api/invoices.

↓

Endpoint -> dispatcher -> handler
The endpoint validates; the dispatcher resolves the command handler.

↓

Handler -> domain -> repository
The handler checks the customer, constructs the invoice, and stages it.

↓

Unit of work -> response
SaveChangesAsync commits the write; the API returns the new invoice ID.

SAY THIS
A command is a request to change the business state. Its handler organizes the work. The invoice object protects its own rules. The repository stages persistence, and the unit of work saves the change. The controller and endpoint mainly translate between HTTP and the application.

Show in order: InvoicesController.Create; InvoiceApiClient.CreateInvoiceAsync; InvoicesEndpoints.CreateInvoiceAsync; Dispatcher.SendAsync; CreateInvoiceCommandHandler.HandleAsync; Invoice.Create; UnitOfWork.SaveChangesAsync. [T06:74] [T07:68] [T08:99] [T09:76] [T12:14]

Explain failure as data

Result and Result<T> contain success or a business error. For example, a missing customer or invalid invoice dates can return a failure instead of completing a save. The normal REST endpoint turns these outcomes into appropriate HTTP responses. Unexpected infrastructure exceptions follow a different exception path.

ASK: What is the difference between adding an entity and saving it?

LISTEN FOR: InvoiceRepository.AddAsync stages the entity in EF Core. UnitOfWork.SaveChangesAsync performs the save. [T11:32]

One important validation detail
The REST endpoint explicitly calls validator.ValidateAsync. The AI dispatcher path does not run that validator automatically. Domain rules still run, but do not teach validator registration as universal enforcement. [T08:112] [T10:20]

08 / INSTRUCTOR GUIDE

Explain queries using the dashboard

Queries read information; commands request a state change. This app uses different persistence approaches for those jobs.

SAY THIS
To create or update an invoice, we work with business objects and save changes through EF Core. To show a summary, we use a SQL query through Dapper that returns the data the screen needs. Both reach the same business database, but they serve different tasks.

```
GetInvoiceSummaryQuery
 -> GetInvoiceSummaryQueryHandler.HandleAsync
 -> IInvoiceReadRepository.GetSummaryAsync
 -> Dapper aggregate SQL -> InvoiceSummaryDto
```

Current metric	Meaning in the implemented query
TotalInvoiceAmount	Sum across all invoices, including Cancelled.
TotalPaidAmount	Full amounts for invoices whose status is Paid.
TotalOutstandingAmount	Amounts for Draft, Sent, and Overdue invoices.
OverdueInvoices	Count with stored status Overdue.
Overdue list	Due date before today in UTC; excludes Paid and Cancelled.

The summary is an all-time snapshot. Its SQL does not filter by currentDate even though the repository method accepts that argument. The due-date overdue list can contain a past-due Sent or Draft invoice whose stored status is not Overdue. [T13:19] [T14:188] [T14:158]

Make students read the units

On the dashboard, the Overdue card displays a count, while Outstanding displays an amount. Ask students to identify the data property before interpreting the card. "Paid amount" is inferred from invoice status; this path does not total bank transactions or partial payments. [T40:64]

ASK: Can the Overdue status count differ from the due-date overdue list count?

LISTEN FOR: Yes. One checks stored status; the other checks date and excludes Paid/Cancelled. They answer different implemented questions.

Use code to settle metric definitions
Some Markdown descriptions exclude cancelled invoices or drafts from totals differently. Show the SQL and explain that the current implementation is the behavior being demonstrated. Treat the discrepancy as a documentation improvement exercise.

09 / INSTRUCTOR GUIDE

Introduce AI from the endpoint

Now replace a fixed form request with a natural-language request, while keeping the business foundation visible.

> **SAY THIS**
> The chat entry point does not calculate the balance itself. It accepts a message and asks the coordinator to handle it. The response includes an answer and information about which agent responded. Underneath, the agent can use the business operations we already studied.

```
POST /api/ai/chat
Content-Type: application/json

{
  "message": "Summarize invoice activity",
  "conversationId": "lesson-summary-01",
  "context": { "page": "Dashboard" }
}
```

Show MapAiEndpoints. The endpoint rejects a blank message, creates an eight-character ID when no conversation ID is supplied, awaits coordinator.ProcessAsync, attaches suggestions by agent name, and returns AiChatApiResponse. [T15:21]

Response field	What students should understand
message / agentName	The answer and the display name of the selected agent.
conversationId	The value the caller reuses for conversational history.
isSuccess	The agent execution result, not a guarantee that every requested business action succeeded.
suggestions	Static follow-up strings selected by agent name.
confidenceScore / durationMs	Heuristic confidence and agent-reported timing, each with implementation limits.

Distinguish the two HTTP routes

The browser posts to **/ai/chat** on InvoiceApp.Web, whose AiController uses an antiforgery-protected action. The web assistant service and API client then post to **/api/ai/chat** on InvoiceApp.Api. Calling the backend endpoint directly bypasses the MVC proxy, not the coordinator. [T28:44]

ASK: Why does the endpoint accept a conversation ID?

LISTEN FOR: So later messages can retrieve earlier turns. The ID does not itself prove a user identity or permission.

10 / INSTRUCTOR GUIDE

Explain the coordinator as a workflow

Use the receptionist analogy only long enough to make the ordered C# steps memorable.

SAY THIS
The coordinator receives the question, retrieves recent conversation text, chooses one specialist, waits for its reply, evaluates the reply, and saves the interaction. It is orchestration code. In this app, the coordinator itself does not call a language model to choose the specialist.

```
history -> enriched prompt
RouteToAgent(latestUserPrompt)
 -> targetAgent.ExecuteAsync(agentRequest)
 -> EvaluateResponseAsync(originalPrompt, response.Content)
 -> save user and agent messages
 -> return CoordinatorResult(response, evaluation)
```

Priority	Matching text	Selected agent
1	how do i; how to; how can i; guide; documentation; rule; can i delete	Documentation
2	create customer; add customer; update customer; find customer	Customer operations
3	kpi; revenue; summary; activity; analytics; who owes	Analytics
4	Everything else	Invoice operations

The first matching branch wins, using the latest message only. It is substring matching rather than model-based semantic classification. The customer branch has an outer customer/client/onboard check, then requires an operation phrase. [T16:72] [T16:111]

ASK: Predict "How do I create customer records?" and "Which customer owes the most?"

LISTEN FOR: The first goes to documentation. The second falls through to invoice operations, because it does not contain who owes.

Memory is useful but limited

The memory service returns ten individual messages, joins them into text, and the coordinator prepends them to the latest prompt. It retains older entries in the stored list and loses history on process restart. History influences execution text but not routing. [T26:37]

A teachable follow-up
After an analytics turn, ask students to predict "And last month?" It routes to invoice operations because no analytics phrase matches. Do not describe this router as understanding the whole conversation.

11 / INSTRUCTOR GUIDE

Explain model, agent, and tool separately

These three words describe different parts of the request path.

Term	Concrete meaning in this app
Model / IChatClient	The configured chat model receives messages and can request registered functions.
Agent / BaseAgent subclass	A C# wrapper with instructions, name, description, tools, and a simulation override.
Tool / AIFunction	A C# function the model can request, such as GetInvoiceSummary.

SAY THIS
The model can ask for a tool, but the model does not run SQL by itself here. The .NET function runs inside the application. That function calls a query or command, receives real data or a business outcome, and returns it for the model to explain.

```
// Tool descriptor:
AIFunctionFactory.Create(GetInvoiceSummary, nameof(GetInvoiceSummary),
    "Retrieves global billing summary ...");

// Shared live execution:
new ChatMessage(ChatRole.System, Instructions);
new ChatMessage(ChatRole.User, request.Prompt);
// ChatOptions: Tools = _tools, Temperature = 0.2f
await chatClient.GetResponseAsync(messages, chatOptions, cancellationToken);
```

Descriptions supply tool/argument guidance. UseFunctionInvocation enables automatic execution of requested functions. The model may also answer without using a tool; the code does not force every answer to retrieve data. [T19:48] [T18:89] [T17:84]

Specialist	Current tool access
Invoice operations	7 invoice tools plus 5 customer tools.
Customer operations	5 customer tools.
Analytics	The same 12 invoice/customer tools, including writes.
Documentation	ReadDocumentationPage: reads full Markdown files.

ASK: Does calling it an analytics agent make it read-only?

LISTEN FOR: No. Its registered function list includes write capabilities. Instructions and enforced capabilities are different.

Document retrieval here is file lookup, not vector search. The implementation uses custom orchestration around Microsoft.Extensions.AI and Azure.AI.OpenAI. Avoid presenting the code as proof that a separate Microsoft.Agents SDK workflow is running. [T21:41] [T39:16]

12 / INSTRUCTOR GUIDE

Teach the three execution modes

Identify the mode before asking students to interpret an answer or a side effect.

Mode	What triggers it	What it actually does
Live backend agent	Usable-looking Azure configuration and a non-null chat client.	Calls the model with instructions and functions. Real tools may read/write the database.
Backend simulation	Missing/recognized placeholder configuration, or a null chat client.	Skips model calls. Agent-specific C# branches can still execute real tools.
Web mock fallback	AI API transport/envelope failure or exception in the web assistant service.	Uses templates and ordinary invoice/customer API clients.

SAY THIS
Simulation means we skip the language model, not necessarily the database. A simulated invoice status request can still call the real update tool. The web mock is a separate fallback with different behavior, and it still needs the business API to retrieve useful records.

Backend analytics simulation always returns the live global summary, regardless of the specific analytical question. Customer creation simulation returns guidance instead of creating a customer. Invoice simulation can change a status but can also produce misleading confirmation text around an error JSON result. [T24:70] [T23:68] [T22:73]

Explain the runtime failure case accurately

A bad Azure key in live mode does not automatically activate backend simulation. BaseAgent catches the runtime exception and returns a failed AgentResponse. AiEndpoints returns that envelope as HTTP 200, so the web assistant normally forwards it rather than falling back. [T17:131] [T29:50]

Demonstration discipline
Start a fresh conversation for each independent mode example. Record invoice status before and after any update prompt. Do not use a chat acknowledgment, a green icon, or the Studio tool-history row as the sole proof of a saved operation.

ASK: Can a student demonstrate AI creation of an invoice in backend simulation?

LISTEN FOR: The current invoice override returns creation guidance. They can demonstrate normal UI creation, or explain a possible live tool call, but should not claim simulation created an invoice.

13 / INSTRUCTOR GUIDE

A repeatable AI classroom demonstration

Use these steps in a fresh conversation, and compare predictions with the actual route and mode.

Step / prompt	Show and explain	What to expect
1. "How do I create an invoice?"	Point to the documentation route and its documentation tool.	Documentation agent. Simulation returns creating-invoice text; live model may retrieve it.
2. Fresh chat: "Summarize invoice activity"	Compare the reply with the summary query and dashboard definitions.	Analytics agent. Simulation directly returns global summary JSON; live output wording may vary.
3. Fresh chat: "Show overdue invoices"	Ask whether due date or stored status determines the list.	Invoice operations agent; simulation calls the overdue query.
4. Fresh chat: "Which customer owes the most?"	Predict the route, then inspect it.	Invoice operations, not analytics. No implemented dedicated ranking tool.

A request-level demonstration without the UI

```
# Run this against your training API, using the actual listening URL.
$payload = @{
    message = 'Summarize invoice activity'
    conversationId = 'class-summary-01'
} | ConvertTo-Json
Invoke-RestMethod -Method Post -Uri 'http://localhost:5231/api/ai/chat' `
    -ContentType 'application/json' -Body $payload
```

Use the direct API example after the UI demonstration. Inspect agentName, isSuccess, confidenceScore, and message. Let students explain why a successful HTTP response does not prove an invoice mutation occurred. This example is read-oriented; the document does not execute it.

Optional write demonstration

Use a known disposable training invoice in Sent status and its actual number. Ask "Mark [that number] as Paid." Predict the route and inspect the saved status afterward. In simulation, compare the JSON tool result with the surrounding confirmation. If the invoice is Draft, use the rejection to teach the state machine rather than repeatedly changing the prompt.

> **Keep claims tied to evidence**
> Live model wording and tool choices are not guaranteed. Use backend logs and business records as evidence. If you only have simulation available, describe the live path as code behavior to inspect, not a live demonstration you have completed.

14 / INSTRUCTOR GUIDE

Teach evaluation by challenging a score

This is an opportunity to teach evidence-based AI engineering using a deliberately simple evaluator.

SAY THIS
The evaluator currently checks text patterns. It does not compare the answer with database records. A confidence score is meaningful only if we understand how it was produced. Here, 0.95 is a fixed rule-based value, not a measured probability that the answer is correct.

```
hasRisk = response.Contains("I am not sure", OrdinalIgnoreCase)
          || response.Length < 10;
passedPolicy = !response.Contains("confidential_bypass", OrdinalIgnoreCase);
confidence = hasRisk ? 0.65 : 0.95;
```

AgentEvaluationResult carries the score, policy flag, risk flag, and feedback. Evaluation runs after execution. The coordinator does not reject the response when policy fails, and the API exposes only the confidence score. [T27:44] [T15:50]

Example answer	Current result	Teaching point
"Paid."	Risk=true, confidence=0.65.	A short correct answer can be flagged.
"All balances are zero."	Risk=false, confidence=0.95.	A plausible but wrong answer can score highly.
"Use confidential_bypass now."	Policy=false, confidence=0.95.	Policy and confidence are independent.

ASK: What evidence would establish that an invoice was marked Paid?

LISTEN FOR: A successful domain/tool outcome and the persisted invoice state, or a verified read afterward. Not just a confident natural-language sentence.

Distinguish display from execution evidence

Studio infers tool history from keywords in the prompt and reply. It does not receive a server tool-execution trace. Its UI also substitutes a default confidence when the score is absent or zero. Use the inspector to discuss observability gaps, not to certify a tool ran. [T31:294]

A productive class question
Ask students to propose one improvement: checking a specific factual claim against a tool result, returning structured tool outcomes, or exposing real server execution events. Label it proposed work rather than an existing feature.

15 / INSTRUCTOR GUIDE

Explain failures without losing the class

Teach students to locate the failing boundary before changing code or prompts.

Symptom	Explain and inspect
Blank prompt rejected	Input validation at the chat endpoint; inspect the HTTP response.
Wrong agent answers	Inspect RouteToAgent and the latest message keywords.
Page context seems ignored	It is forwarded in AgentRequest, but BaseAgent does not put Context into the model messages.
HTTP 200 but no action completed	Inspect agent isSuccess and the tool result. Business failure can be returned inside a normal tool string.
Unexpected simulation text	Identify backend simulation versus web mock fallback from configuration/logs.
Database changes despite a later error	Commands save independently; there is no transaction covering the whole chat turn.

SAY THIS
A useful debugging question is: what is the last boundary that worked? Did the browser send the request? Did the endpoint receive it? Which agent was selected? Was a tool called? Did the handler accept the rule? Was the database save completed?

A short breakpoint route

Use AiEndpoints, AgentCoordinator.ProcessAsync, RouteToAgent, BaseAgent.ExecuteAsync, the selected tool, Dispatcher, the handler, and the domain method or read repository. For a write, inspect SaveChangesAsync and then read the record again.

Performance and cancellation: a short advanced discussion

Explain async as waiting for I/O without blocking the calling thread for the entire wait. A CancellationToken lets a request signal it is no longer needed; propagation alone does not guarantee every service observes it. BaseAgent catches live-path cancellation as an Exception, while simulation work occurs outside that catch.

History retrieval is limited to ten messages but storage is not pruned. Concurrent requests for the same conversation can interleave, and memory is not shared across API instances. Simulation duration excludes tool work because its stopwatch stops early. These are concrete limits, not reasons to introduce every distributed-systems concept in the first lesson.

ASK: Should we blindly retry a failed create request?

LISTEN FOR: No. A write may have completed before a later error or lost response. First determine what was saved; retries need an idempotency strategy.

Student exercises: predict, trace, calculate

Give students 15 minutes: approximately 4 minutes for A, 6 for B, and 5 for C. Answers are on the next page.

A. Predict the agent

For each message, name the actual selected agent and the first matching condition. Do not use your intuition about the best specialist.

Message	Student answer
How do I create customer records?	Agent: _____
Show revenue summary	Agent: _____
Show overdue invoices	Agent: _____
Which customer owes the most?	Agent: _____

B. Trace one invoice status update

Start with a disposable training invoice in Draft. Predict whether Draft-to-Paid succeeds. Name the domain method enforcing the result. Then write the valid path to Paid. Identify the tool, command handler, domain check, and persistence boundary. Explain why the same domain rule should hold for a form and an AI tool.

C. Calculate the implemented metrics

This is a paper-only fixture, independent of the NGN 60,000 classroom invoice and any seeded database. Assume the dates shown are relative to the query date in UTC.

Invoice	Status	Amount (NGN)	Due date
A	Draft	60,000	Future
B	Sent	40,000	Past
C	Paid	30,000	Past
D	Cancelled	20,000	Past
E	Overdue	10,000	Past

Calculate: total invoice count, total invoiced amount, paid amount, outstanding amount, stored Overdue count, and the count/value of the due-date overdue list. State why the two overdue counts differ.

Submission format
One page containing predictions, a request-path diagram, and metric calculations. Ask students to cite at least one method or SQL condition for each explanation. They can use the code; the aim is reasoning, not memorization.

Instructor answer key and debrief

Ask a student to explain the reasoning before revealing each answer.

A. Route predictions

Message	Correct result
How do I create customer records?	Documentation. The how do i rule wins first.
Show revenue summary	Analytics. Revenue/summary matches.
Show overdue invoices	Invoice operations. No earlier rule matches.
Which customer owes the most?	Invoice operations. The analytics rule expects who owes.

B. Status path

Draft-to-Paid fails. Invoice.TransitionTo rejects it. The valid path is Draft-to-Sent-to-Paid. A live AI workflow can use InvoiceTools.UpdateInvoiceStatus, which resolves the invoice ID and dispatches UpdateInvoiceStatusCommand. Its handler loads the invoice, invokes TransitionTo, and calls IUnitOfWork.SaveChangesAsync only on success. The normal HTTP status endpoint reaches the same application/domain operation.

Listen for a distinction between the model choosing an operation and the domain accepting it. A student who says "the prompt prevents it" has not yet located the enforced rule.

C. Paper fixture metrics

Metric	Correct value	Reason
Total invoices	5	All five rows.
Total invoiced	NGN 160,000	60,000 + 40,000 + 30,000 + 20,000 + 10,000; includes Cancelled.
Paid amount	NGN 30,000	Only C is Paid.
Outstanding	NGN 110,000	A + B + E: Draft, Sent, Overdue.
Stored Overdue count	1	Only E has Overdue status.
Due-date overdue list	2 invoices; NGN 50,000	B and E are past due and neither Paid nor Cancelled.

ASK: What changes if A is also past due?

LISTEN FOR: The summary amounts and stored Overdue count stay the same. The due-date list gains A, so it becomes 3 invoices totaling NGN 110,000.

> **Debrief the central lesson**
> The same business words can have different implemented definitions. Good explanations identify the exact method, status, date rule, and unit of measurement behind a number.

Common student questions and honest answers

Use these answers to keep a training example separate from features that would require additional implementation.

Question	Answer you can give
Is the coordinator itself AI?	This coordinator is ordinary C# workflow and keyword routing. The selected live agent calls the model.
Can the model access any table?	It receives registered functions. The inspected tools dispatch specific application operations, not arbitrary SQL.
Are business rules just system prompts?	No. The domain enforces core invoice rules. Prompt instructions guide the model, but do not replace those checks.
Does simulation mean nothing changes?	No. Some backend overrides invoke real tools, including invoice status updates.
Is 95% confidence proof?	No. It is a fixed heuristic score from simple text checks.
Can it create invoices without a model?	Normal UI/API creation works independently of the model. The current invoice-agent simulation returns guidance for creation.
Does overdue status change automatically?	The overdue query checks dates without changing status. Explicit commands change the stored status.
Is there a real audit trail in Studio?	Its tool list is inferred from text. It is not an authoritative server execution history.

Do not repeat these documentation shortcuts

Do not promise gap-free invoice numbers: sequence values can have gaps, and the fallback numbering logic has concurrency limits. Do not claim automatic tax calculation in the inspected item total. Do not claim every registered validator runs for AI tools. Do not describe global summaries as a completed monthly reporting feature.

The current AI path also has no implemented per-user tool authorization or mandatory approval gate for writes. MVC antiforgery is different from authorization. Present these as engineering work to add, rather than teaching the prototype as a complete production security model. [T35:16]

> **Answering a question outside the current implementation**
> Say: "That would be a useful extension. Let us first locate whether it exists in the code, then identify which layer should own it." This encourages investigation without turning planned capabilities into claimed features.

Assess understanding and close the lesson

Have students explain a new scenario rather than repeat your exact wording.

A two-minute student teach-back

Ask each student or pair to explain: "A user asks to mark an invoice Paid. What receives the message, who chooses an agent, what performs the update, where is the rule enforced, and what proves the result?" Give them the whiteboard and ask them to identify one possible failure.

Criterion	0 points	1 point	2 points
Business model	Cannot explain invoice total/status.	Explains part of the calculation or rule.	Explains derived total and valid transitions.
Request flow	Mixes layers or skips the core path.	Mostly correct but misses a boundary.	Traces endpoint, coordinator, tool, handler, domain, save.
AI boundaries	Assumes model executes database rules.	Separates model and tool partly.	Separates model choice, tool execution, and domain enforcement.
Evidence and limits	Treats reply/score as proof.	Mentions one caveat.	Checks outcome/state and identifies a mode or evaluation limit.

Suggested interpretation: 7-8 points = ready to explain independently; 5-6 = targeted practice; 0-4 = revisit the business example and trace one request together. This is a proposed classroom rubric, not a validated certification score.

Optional extension assignment

Choose one scoped improvement: add an evaluator test exposing a false-positive score; improve one routing phrase and test precedence; design a structured server tool-execution record; or move shared validation into an application boundary. Require a before/after example, an explanation of the changed layer, and focused validation. These are student tasks, not changes made by this guide.

Use InvoiceTests for arithmetic and state-machine examples. AiTests covers tool registration and limited simulation/routing cases. Existing integration tests use WebApplicationFactory and replace repositories with test doubles, so they do not demonstrate real SQL Server or live Azure behavior. [T32:8] [T33:14] [T34:123]

> **SAY THIS**
> We started with a billing problem and ended with an AI-assisted way to use the same application. The endpoint accepts input, the coordinator organizes the turn, the model can choose tools, and the application and domain decide what can actually happen. A good engineer checks the evidence behind both an operation and an answer.

20 / INSTRUCTOR GUIDE

Teaching source map: business and backend

References such as [T01:147] identify a repository file and exact line. Paths are relative to InvoiceAiApp; line numbers describe the reviewed workspace snapshot.

ID	File and reading location
T01	src/InvoiceApp.Domain/Entities/Invoice.cs Referenced lines: 23, 25, 102, 147
T02	src/InvoiceApp.Domain/Entities/InvoiceItem.cs Referenced lines: 13
T03	src/InvoiceApp.Domain/Entities/Customer.cs Referenced lines: 7
T04	src/InvoiceApp.Api/Program.cs Referenced lines: 30, 44, 61
T05	src/InvoiceApp.Web/Program.cs Referenced lines: 31
T06	src/InvoiceApp.Web/Controllers/InvoicesController.cs Referenced lines: 74
T07	src/InvoiceApp.Web/Services/ApiClients/InvoiceApiClient.cs Referenced lines: 68
T08	src/InvoiceApp.Api/Endpoints/InvoicesEndpoints.cs Referenced lines: 99, 112
T09	src/InvoiceApp.Application/Features/Invoices/Commands/CreateInvoice/CreateInvoiceCommand.cs Referenced lines: 76
T10	src/InvoiceApp.Application/Common/Messaging/Dispatcher.cs Referenced lines: 20
T11	src/InvoiceApp.Infrastructure/Persistence/Repositories/InvoiceRepository.cs Referenced lines: 32
T12	src/InvoiceApp.Infrastructure/Persistence/Repositories/UnitOfWork.cs Referenced lines: 14
T13	src/InvoiceApp.Application/Features/Invoices/Queries/GetInvoiceSummary/GetInvoiceSummaryQuery.cs Referenced lines: 19
T14	src/InvoiceApp.Infrastructure/Persistence/Repositories/InvoiceReadRepository.cs Referenced lines: 158, 188
T15	src/InvoiceApp.Api/Endpoints/AiEndpoints.cs Referenced lines: 21, 50
T16	src/InvoiceApp.AI/Coordinator/AgentCoordinator.cs Referenced lines: 72, 111
T17	src/InvoiceApp.AI/Agents/BaseAgent.cs Referenced lines: 84, 131
T18	src/InvoiceApp.AI/DependencyInjection.cs Referenced lines: 89
T19	src/InvoiceApp.AI/Tools/InvoiceTools.cs Referenced lines: 48
T20	src/InvoiceApp.AI/Tools/CustomerTools.cs Supporting source; inspect the type and related methods.
T21	src/InvoiceApp.AI/Tools/DocumentationTools.cs Referenced lines: 41

Recommended code-tour order: domain invoice and item -> normal invoice endpoint -> create handler -> persistence -> AiEndpoints -> coordinator -> BaseAgent -> InvoiceTools -> evaluator. Use the IDs above to prepare tabs in that order.

21 / INSTRUCTOR GUIDE

Teaching source map: AI, UI, and tests

Use implementation and focused tests to substantiate the lesson. Treat comments, README curriculum claims, and user-facing documentation as descriptions to verify.

ID	File and reading location
T22	src/InvoiceApp.AI/Agents/InvoiceAssistantAgent.cs Referenced lines: 73
T23	src/InvoiceApp.AI/Agents/CustomerOperationsAgent.cs Referenced lines: 68
T24	src/InvoiceApp.AI/Agents/AnalyticsAgent.cs Referenced lines: 70
T25	src/InvoiceApp.AI/Agents/DocumentationAgent.cs Supporting source; inspect the type and related methods.
T26	src/InvoiceApp.AI/Memory/AgentMemoryService.cs Referenced lines: 37
T27	src/InvoiceApp.AI/Evaluations/AgentEvaluationService.cs Referenced lines: 44
T28	src/InvoiceApp.Web/Controllers/AiController.cs Referenced lines: 44
T29	src/InvoiceApp.Web/Services/Ai/AgentFrameworkAiAssistantService.cs Referenced lines: 50
T30	src/InvoiceApp.Web/Services/Ai/MockAiAssistantService.cs Supporting source; inspect the type and related methods.
T31	src/InvoiceApp.Web/wwwroot/js/studio-chat.js Referenced lines: 294
T32	tests/InvoiceApp.UnitTests/Domain/InvoiceTests.cs Referenced lines: 8
T33	tests/InvoiceApp.UnitTests/AI/AiTests.cs Referenced lines: 14
T34	tests/InvoiceApp.IntegrationTests/ApiEndpointsTests.cs Referenced lines: 123
T35	src/InvoiceApp.Infrastructure/Services/SqlServerInvoiceNumberGenerator.cs Referenced lines: 16
T36	src/InvoiceApp.Web/Properties/launchSettings.json Referenced lines: 8
T37	src/InvoiceApp.Api/Properties/launchSettings.json Referenced lines: 8
T38	src/InvoiceApp.Web/appsettings.Development.json Referenced lines: 9
T39	src/InvoiceApp.AI/InvoiceApp.AI.csproj Referenced lines: 16
T40	src/InvoiceApp.Web/Views/Dashboard/Index.cshtml Referenced lines: 64

Preparation record

Source files and cited lines were resolved during PDF generation. The document layout, extracted text, page count, and bookmarks were checked. Lesson timings and exercises are teaching recommendations; runtime demonstrations and tests should be rehearsed in your training environment.